FACULTY OF SCIENCES OF SFAX — UNIVERSITY OF SFAX

An Agentic Framework for IaC Misconfiguration Analysis and Automated Remediation

Ahmedou Yahye Kheyri

Supervised by: **Prof. Hala Bezine**

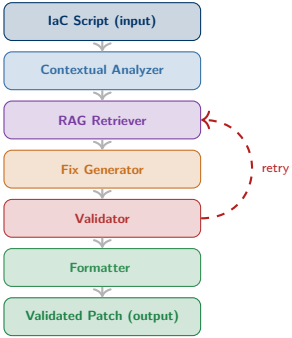
Master MRSI — PFE 2025–2026

Progress Report 2 — April 2026

IaC Security · RAG · LLMs · Checkov

Outline

1. Context & Motivation
2. State of the Art
3. Framework Architecture
4. Evaluation Dataset
5. Evaluation Methodology
6. Experimental Results
7. Implementation Status
8. Conclusion & Perspectives



1. Context & Motivation

Why does IaC security matter?

Infrastructure as Code (IaC): The New Attack Surface

What is IaC?

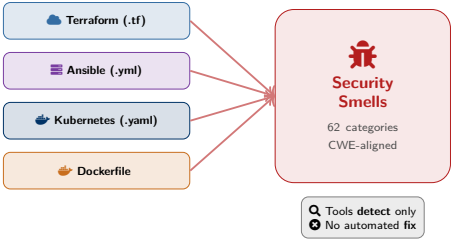
Code that **defines and provisions** cloud infrastructure — servers, networks, databases, firewalls.

4 dominant formats:

- **Terraform** (.tf) — AWS, Azure, GCP
- **Ansible** (.yml) — configuration mgmt
- **Kubernetes** (.yaml) — container orchestration
- **Dockerfile** — container images

The Problem

IaC scripts are shipped with **security misconfigurations**. **62 categories** of security smells (War et al., 2025): hard-coded secrets, open firewall rules, privileged containers...



Research Gap & Our Contribution

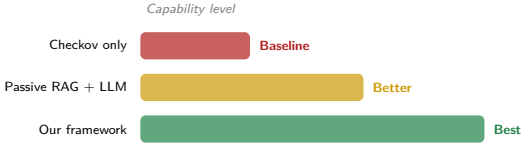
Limitations of Existing Approaches

- **Checkov / Terrascan**: detect only, zero fix generation
- **Passive RAG systems**: patches with no validation
- **LLM hallucinations**: wrong patches, no safety net
- **No iterative loop**: one shot, fail silently

Our Proposed Approach

- LLM generation + **external validation**
- **CRAG** (Corrective RAG) — refines on failure
- **Retry loop** (max 3 attempts)
- Validator = Checkov (independent judge)
- Target output: validated diff + CWE explanation

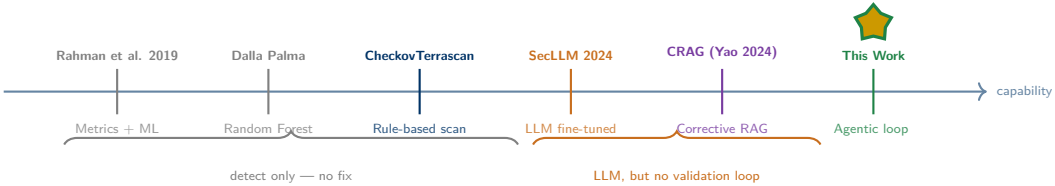
> *Can we close the loop: detect → retrieve → generate → validate?*



2. State of the Art

From detection to remediation

Related Work at a Glance

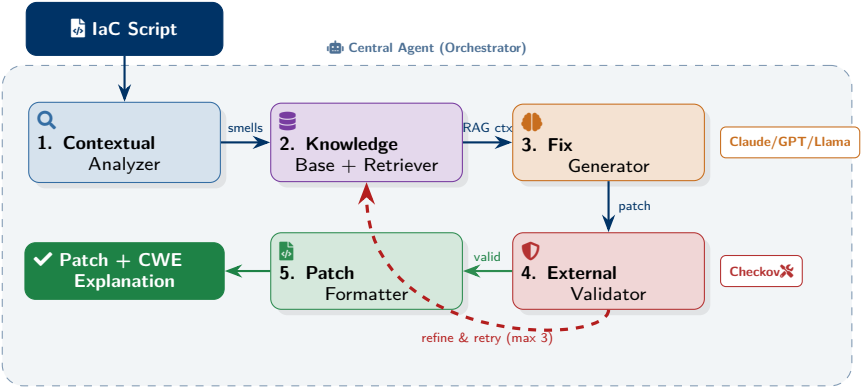


Work	Approach	Coverage	Fix?	Validation?
War et al. 2025	62-cat taxonomy	Multi-tool	✗	✗
SecLLM 2024	LLM fine-tuned	Terraform	✓	✗
GenSIAC 2024	Fine-tuned GPT	IaC patches	✓	✗
CRAG (Yao)	Corrective RAG	General NLP	✗	✗
This work	**Agentic RAG+LLM**	**4 tools**	✓	✓

3. Framework Architecture

6 modules — 1 agentic loop

System Architecture — The Big Picture



Module 1 – Contextual Analyzer

Role & Approach

1. Detect **IaC tool type** (extension + regex)
2. Run **Checkov** → parse JSON failures
3. Extract **metrics**: lines, tokens, blank/comment count
4. **Fallback**: regex heuristics if Checkov unavailable

Heuristic smell patterns (fallback)

- hardcoded password CWE-259
- open CIDR 0.0.0.00 CWE-732
- privileged container CWE-250
- TLS disabled CWE-295

Listing 1: Tool detection logic

```python
TOOL_SIGNATURES = {
  "terraform": [
    r'resource\s+"',
    r'provider\s+"',
    r'terraform\s*\{'],
  "kubernetes": [
    r"apiVersion:",
    r"kind:\s+Deployment"],
  "ansible": [
    r"^\s*-\s+name:",
    r"^\s*hosts:"],
  "docker": [
    r"^FROM\s+",
    r"^RUN\s+"],
}
# Then: subprocess.run(["checkov",
#   "--file", str(path),
#   "--output", "json", "--quiet"])
```

Modules 2 & 3 – Knowledge Base & RAG Retriever

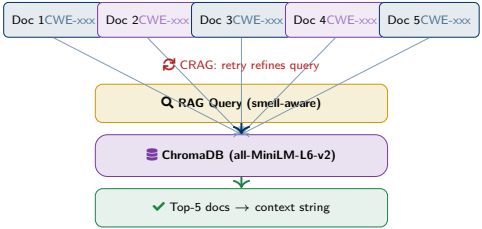
Knowledge Base (ChromaDB)

- **62-category** taxonomy (War et al., 2025)
- Embeddings: `all-MiniLM-L6-v2` (80MB, free)
- Each document: name + category + description + CWE + fix example

CRAG Retrieval Strategy

Try	Query Augmentation
1	Direct smell descriptions
2	+ "minimal targeted changes"
3	+ "most conservative fix"

Each retry gets more conservative to break the LLM out of its failure mode.



Module 4 – Fix Generator (LLM)

Multi-Backend LLM Support

Env Var	Backend	Default model
`ANTHROPIC_...`	Anthropic	claude-sonnet-4-6
`OPENROUTER_...`	OpenRouter	llama-3.1-8b:free
`MINIMAX_...`	MiniMax	MiniMax-Text-01
`OPENAI_...`	OpenAI	gpt-4o-mini

Auto-detected from env, **no code change** needed.

Confidence Filtering (SecLLM)

LLM appends: `CONFIDENCE: 0.0-1.0`
Threshold ≥ 0.6 to proceed.
Temperature $= 0.2$ for determinism.

Listing 2: Prompt sent to LLM

```
[SYSTEM]
You are an IaC security expert.
Output ONLY unified diff format.
Minimal change. No refactoring.
Then: CONFIDENCE: 0.0-1.0

[USER]
## IaC Tool: terraform
## Detected Smells:
- Line 5: hardcoded_credential
  [CWE-798]
## Relevant Security Knowledge:
[Doc 1] CWE=CWE-798
  Smell: Hardcoded Credentials
  Fix: Use env vars or Vault...
## Original Script:
resource "aws_instance" "app" {
  access_key = "AKIA1234..."
}
```

Module 5 – External Validator (The Key Innovation)

Why This Module is Critical

This is what makes the framework **active**, not passive.

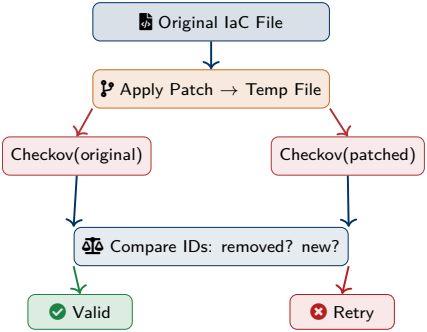
The LLM output is **never trusted** without verification by an independent, certified static analysis tool.

Validation Criterion

Patch is **valid** iff:

$$\text{smell_ids} \subseteq \text{removed} \ \wedge \ \text{new_issues} = \emptyset$$

- All targeted Checkov IDs must disappear
- No new failures introduced



4. Evaluation Dataset

8 files · 27 labelled smells · 4 IaC tools

Dataset Composition & Coverage

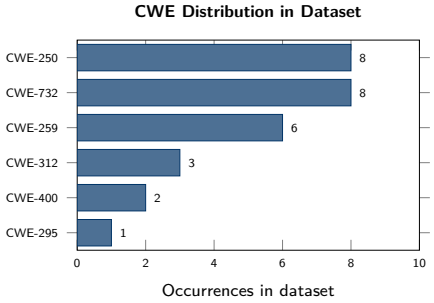
Dataset v1.0

Tool	Files	Smells	%
Terraform	3	11	41%
Ansible	2	7	26%
Kubernetes	2	8	30%
Docker	1	5	19%
Total	**8**	**27**	100%

Each smell: CWE + Checkov ID + line number

Quality Checks

- All smells in 62-category taxonomy
- Manually verified with Checkov
- 40+ Checkov detection tests pass



CWE-250: Unnecessary Privileges · CWE-732: Incorrect Permission Assignment · CWE-259: Hard-coded Password

Dataset: Terraform S3 Example

Listing 3: insecure_s3.tf — 4 smells

```
resource "aws_s3_bucket" "data" {
  bucket = "my-insecure-bucket"
  acl    = "public-read"  # CWE-732
  # Missing: encryption,
  # versioning, access block
}
resource "aws_s3_bucket_policy" "d" {
  policy = jsonencode({
    Statement = [{
      Principal = "*" # open to all!
      Action    = "s3:*"
    }]
  })
}
```

Ground-truth smells — insecure_s3.tf

- **overly_permissive_acl** — CWE-732 · CKV_AWS_20
- **missing_encryption** — CWE-312 · CKV_AWS_145
- **versioning_disabled** — CWE-284 · CKV_AWS_21
- **public_access_block** — CWE-732 · CKV2_AWS_6

All 58 Tests Pass

`pytest tests/ -v` — **58 passed** in 3m29s

5. Evaluation Methodology

4 layers · 18 metrics · ablation study

4-Layer Evaluation Framework

Layer 1 – Detection — Precision · Recall · F1 · Macro-F1 per tool & smell type

Layer 2 – Retrieval — Hit Rate@1 · Hit Rate@3 · Hit Rate@5 · MRR@5 · RAGAS

Layer 3 – Remediation — PVR (Patch Validity Rate) · SER · NNIR · Confidence Calibration

Layer 4 – Agentic Loop — FA-SR (First-Attempt) · ΔR@2 · ΔR@3 · Fleiss' κ

Ablation Study

Config	Description
A	Checkov only (no LLM) — *baseline*
B	LLM only, no RAG
C	LLM + passive RAG (no retry)
D	**Full pipeline (CRAG + Validator)**

Research Questions

- RQ1: Detection accuracy vs ground truth?
- RQ2: RAG retrieval quality?
- RQ3: Patch validity + smell elimination?
- RQ4: Retry loop benefit (ΔR@2, ΔR@3)?

6. Experimental Results

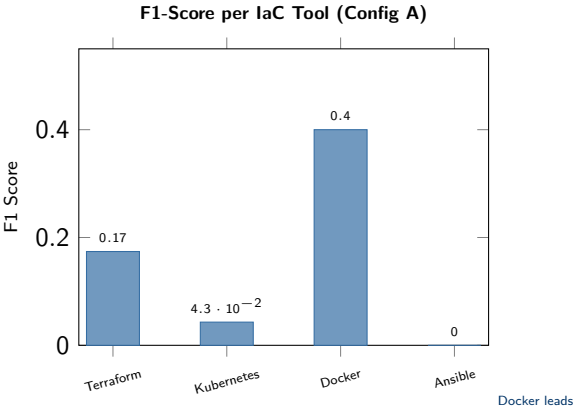
Config A baseline — Checkov detection analysis

Layer 1 – Detection Results (Config A)

Overall Metrics

Metric	Value	Target
True Positives	7	—
False Positives	76	—
False Negatives	16	—
Precision	0.084	> 0.85
Recall	0.304	> 0.85
F1-Score	0.132	> 0.80
Macro-F1	0.082	> 0.80

Low precision: Checkov detects 83 checks vs only 23 in ground truth. This is **expected** — Config D will improve this.



Docker leads (F1=0.40). Ansible=0: smells lack Checkov IDs. ⇒ GLITCH needed for Ansible.

Layers 2, 3 & 4 – Retrieval and Pending

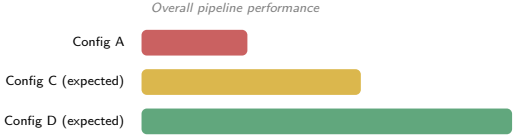
Layer 2 – Retrieval

Metric	Value	Target
Hit Rate @ K=1	**1.000**	> 0.60
Hit Rate @ K=3	**1.000**	> 0.70
Hit Rate @ K=5	**1.000**	> 0.80
MRR @ K=5	**1.000**	> 0.70
Total queries	32	—

Keyword-based approximation over the 62-entry taxonomy. Full RAGAS metrics require ChromaDB integration.

Layers 3 & 4 — Pending (LLM)

Metric	Config A	Config D target
PVR	0.000	> 0.70
SER	0.000	> 0.80
NNIR	0.000	> 0.90
FA-SR	0.000	> 0.50
ΔR@3	0.000	> 0.10



7. Implementation Status

modules under development · baseline results · next steps

Progress Overview

RAG Retriever *Done*	**Knowledge Base** *Done*	**Contextual Analyzer** *Done*
Patch Formatter *In Progress*	**External Validator** *Testing*	**Fix Generator** *In Progress*

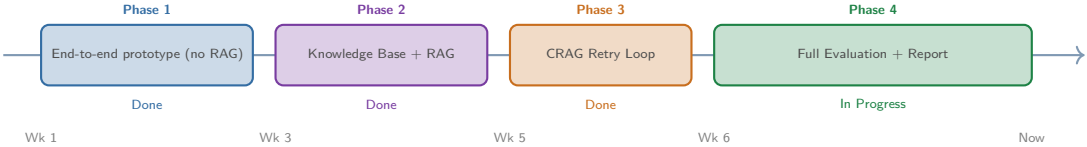
Progress So Far

- **3/6 modules** fully implemented; 3 under testing
- **58 unit tests** passing (3m29s)
- Dataset built: 8 files, 27 labelled smells, 62-cat taxonomy
- Evaluation script: baseline mode running
- Baseline results (Config A) computed and analysed

Planned Next Steps

- Run **Config D** — will test LLM fix generation
- Populate ChromaDB; plan to measure RAGAS metrics
- Explore **GLITCH** for Ansible/Chef smells
- Plan ablation study (B, C, D comparison)
- Consider fine-tuning on GenSIAC dataset

Implementation Timeline



Listing 4: Running the evaluation

```
# Install
pip install -r requirements.txt

# Baseline (no API key needed)
python scripts/evaluate.py \
  --mode baseline

# Full evaluation
export ANTHROPIC_API_KEY="sk-..."
python scripts/evaluate.py \
  --mode full \
  --model claude-sonnet-4-6
```

Technology Stack

Python 3.13 · pytest (58 tests)
ChromaDB · sentence-transformers
Checkov · unified diff
Anthropic SDK · OpenAI SDK

OpenRouter (free Llama/Mistral)

8. Conclusion & Perspectives

Contributions & Future Work

Contributions (In Progress)

1. **Proposed agentic framework** aiming to close the
   detect → retrieve → generate → validate loop for IaC security
2. **CRAG-based strategy** for IaC: query refinement guided by validator feedback (under evaluation)
3. **4-layer evaluation plan** with 18 metrics (detection, retrieval, remediation, agentic)
4. **Curated dataset**: 8 files, 27 smells, 62-category taxonomy

Design Goals

- External validator as the **judge** (planned)
- 4 IaC tools in one framework
- Adaptive RAG on retry
- 4 LLM backends, no lock-in (prototype tested)

Remaining Work & Perspectives

- Complete LLM integration (Configs B, C, D)
- Explore GLITCH for Ansible/Chef
- Consider fine-tuning on GenSIAC
- Expand with real GitHub IaC repos

Thank You

Ahmedou Yahye Kheyri

Master MRSI — PFE 2025–2026

Supervised by: **Prof. Hala Bezine**

Faculty of Sciences of Sfax

April 2026

Questions?

- 3 modules done
- 58 tests pass
- 62-cat taxonomy
- 27 labelled smells
- 4 IaC tools